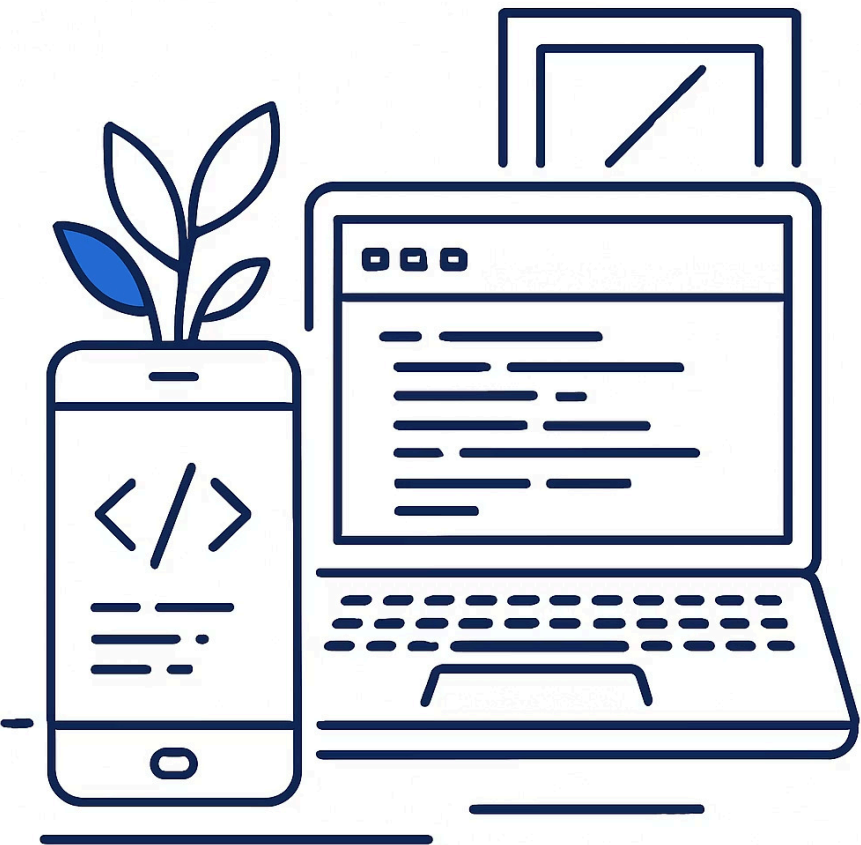




Desarrollo Multiplataforma con React Native

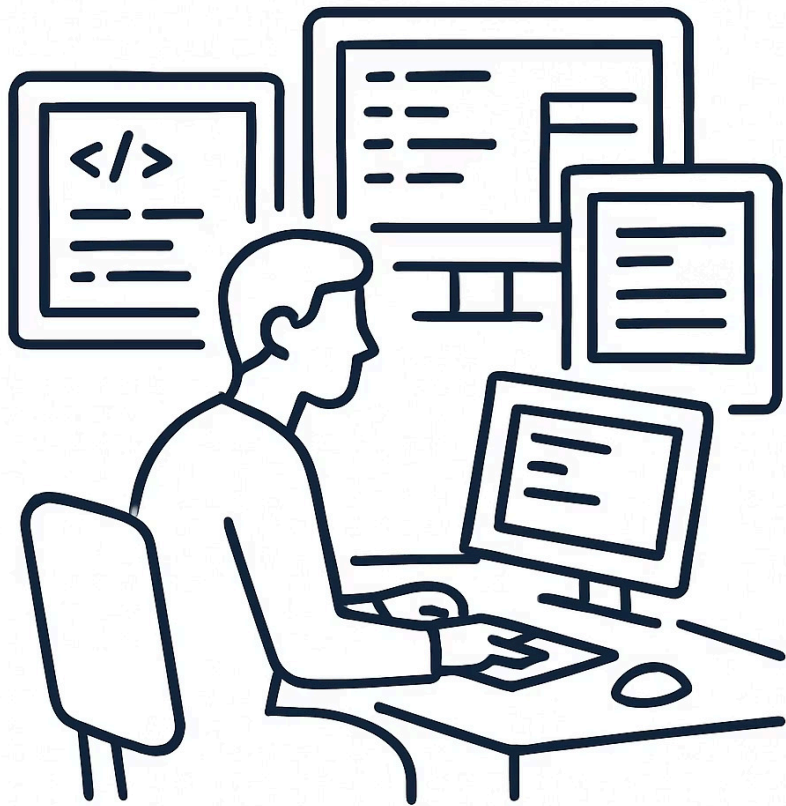
Programa completo del curso · De cero a app publicada

6 MÓDULOS

17 TOPICS

1 APP PUBLICADA

Carlos Rodríguez Contreras · carlos.mailhub@gmail.com · GitHub: karrocon



Sobre el instructor

Carlos Rodríguez Contreras

Senior Software Engineer con más de 15 años transformando ideas en productos digitales reales.



@Shopify (4+ años)

Senior Software Engineer en uno de los mayores referentes globales de e-commerce.



PhD en Informática

Especialidad en Automática. Base académica sólida combinada con experiencia industrial real.



Stack tecnológico

JavaScript · React Native · TypeScript · Ruby · Go

Contenido de esta presentación

01

Presentación del curso

Qué aprenderás, por qué React Native y datos clave del programa.

02

Lo que construiremos

El proyecto final: una app móvil completa con API real y publicación en móvil.

03

Tecnología del curso

El stack completo: TypeScript, React Native, Expo y herramientas de desarrollo.

04

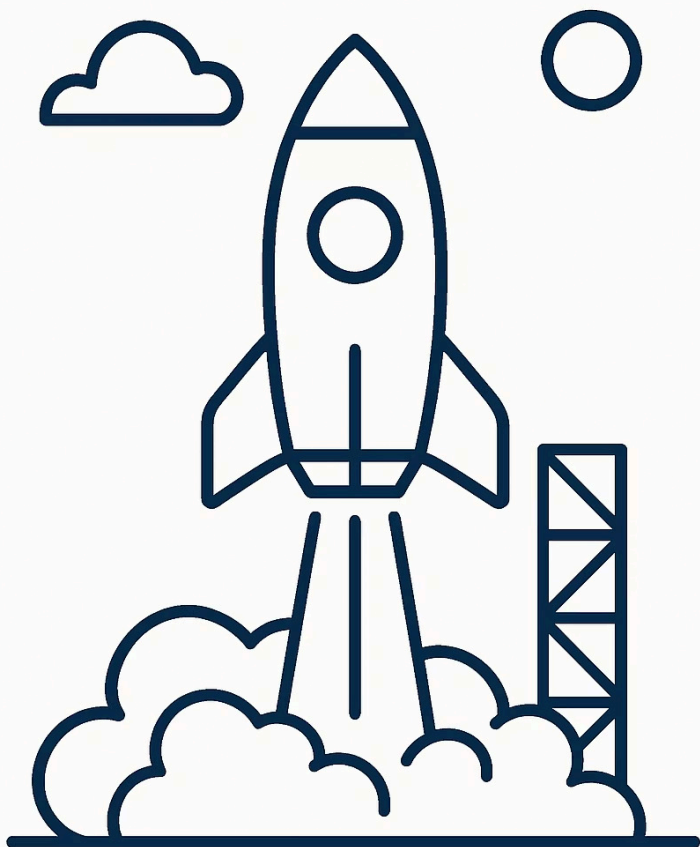
Estructura — 6 módulos

De fundamentos a deploy, módulo a módulo con resultados concretos.

05

Evaluación y metodología

Cómo trabajaremos, criterios de evaluación y materiales disponibles.



MÓDULO 0

Presentación del curso

¿Qué es esto y para qué sirve?

¿Qué aprenderás en este curso?

→ Apps reales para Android e iOS

Desde cero hasta publicación, creando una app funcional que corre en dispositivos reales.

→ TypeScript moderno

Tipos, interfaces y genéricos aplicados directamente a componentes React Native.

→ Componentes y navegación

El modelo de componentes React, navegación entre pantallas y arquitectura de una app real.

→ APIs y datos reales

Conectar la app a servicios externos con Fetch, useEffect y hooks personalizados.

Datos del curso

6

Módulos

17

Topics

Nivel: Iniciación

Requisitos: JS/TS básico



Cada topic incluye ejercicios prácticos que puedes completar a tu ritmo.

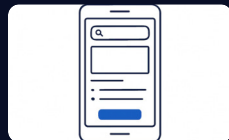
¿Por qué React Native?

Escribe **una sola vez en TypeScript** y obtén apps nativas para Android e iOS. Mantenido por **Meta**, usado en producción por las apps más descargadas del mundo.



Android nativo

Kotlin – solo Android



iOS nativo

Swift – solo iOS



React Native

TypeScript – **Ambas plataformas**



Flutter

Dart – Ambas plataformas

Apps reales construidas con React Native

Empresas líderes en sus sectores confían en React Native para sus apps móviles en producción con millones de usuarios.

Facebook

La app original que motivó la creación de React Native.

Instagram

Partes clave de la app usan React Native en producción.

Shopify

Su app móvil para comerciantes está construida con RN.

Discord

Comunicación en tiempo real con millones de usuarios.

Coinbase

Finanzas críticas sobre infraestructura React Native.

✓ Alta demanda laboral – dominar React Native abre puertas en empresas de todos los tamaños.

PROYECTO FINAL

Lo que construiremos

El proyecto final del curso, paso a paso



El proyecto del curso

Funcionalidades que incluirá tu app

→ Lista de datos con API real

Pantalla principal que consume datos de una API pública, con paginación y rendimiento optimizado.

→ Pantalla de detalle

Navegación fluida entre lista y detalle, con parámetros tipados en TypeScript.

→ Buscador y filtro

Experiencia de usuario completa con búsqueda en tiempo real.

→ Tema claro / oscuro

Preferencias persistidas con AsyncStorage entre sesiones.

→ APK publicado y compartible

Build real generado con EAS Build, distribuible por QR.

¿Qué temática elijo?

- 🎮 Pokédex (referencia del curso)
- 🎬 Películas / series
- 🎮 Videojuegos o deportes
- 🌐 Cualquier API pública
- 💡 **Tú decides**

La temática es libre. El requisito es la funcionalidad, no el tema.

Diseña TU app antes de escribir código

❗ Mini-ejercicio – Toma papel y lápiz **antes** de tocar el teclado. Este paso marca la diferencia entre un proyecto bien estructurado y uno que se refactoriza constantemente.

1

Nombre y temática

¿De qué va tu app? Define el propósito en una sola frase.

2

Pantallas

Dibuja 3-4 rectángulos: *ListaScreen*, *DetalleScreen*, *AjustesScreen*

3

Navegación

Flechas entre pantallas. ¿Cuál es la pantalla de inicio?

4

Componentes

¿Qué piezas se repiten? *ItemCard*, *CategoryBadge*, *DetailHeader*...

5

Modelo de datos

id, nombre, imagen, categoría... define la forma de tus datos.

6

API

¿Qué URL usarás? PokéAPI como referencia si no tienes una todavía.

Los nombres de componentes importan

Un buen nombre de componente describe el **negocio**, no la tecnología. Si alguien lee el nombre, debe entender qué hace sin abrir el archivo.

⊗ **×** Nombres que no dicen nada

Component1

Box2

MyContainer

¿Qué hacen? Imposible saberlo sin leer el código.

✓ **✓** Nombres descriptivos del negocio

PeliculaCard

ActorBadge

GeneroChip

Referencia PokeApp: PokemonCard, TypeBadge, StatsHexagon

Tu proyecto: un repo, 6 módulos

Creas el repositorio en el **Módulo 1** y no vuelves a crear otro. Cada módulo añade nuevas funcionalidades al mismo proyecto, igual que en el trabajo real.



  **PokeApp** es la app de referencia del curso. Tu app puede tener una temática diferente, pero sigue la misma estructura modular.

API de referencia: PokéAPI

La API que usaremos como referencia

<https://pokeapi.co/api/v2/>

Endpoint	Datos
/pokemon	Lista paginada
/pokemon/{id}	Detalle completo
/type/{id}	Tipos disponibles

✓ Sin registro · Sin límite · Gratis. Ideal para aprender.

Ejemplo de respuesta

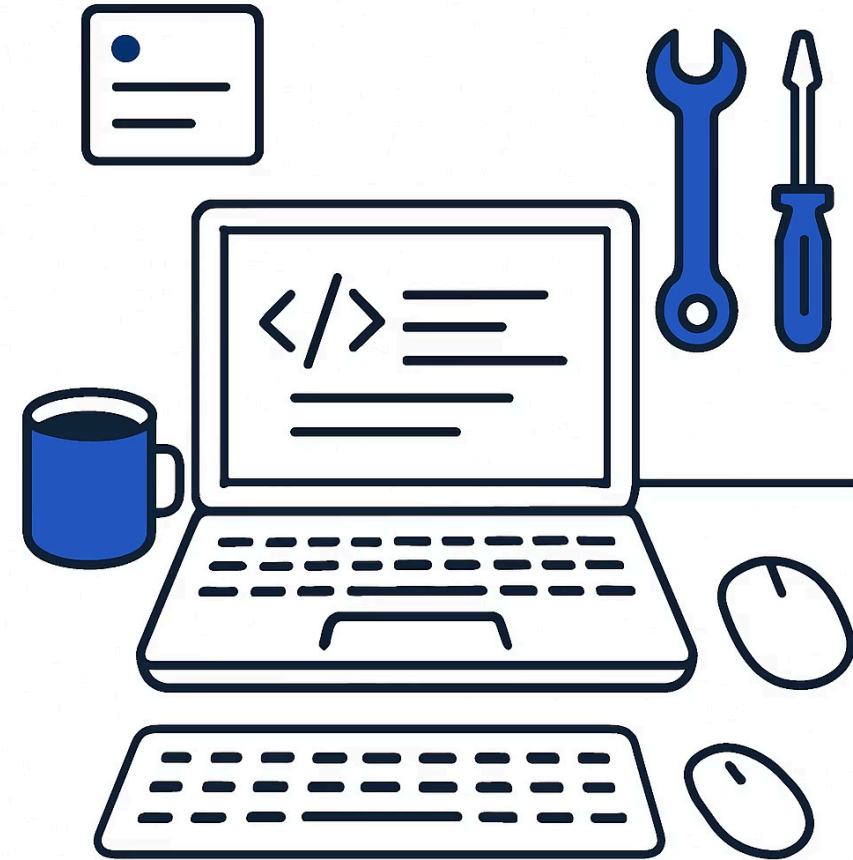
```
GET /pokemon/1
{
  "name": "bulbasaur",
  "id": 1,
  "height": 7,
  "weight": 69,
  "sprites": { ... }
}
```

Respuesta predecible, bien documentada y perfecta para practicar tipado TypeScript.

STACK

Tecnología del curso

Herramientas que necesitas configurar en T01



Stack tecnológico del curso

{ TypeScript

Superset tipado de JavaScript. Añade tipos, interfaces y genéricos que hacen el código más seguro y mantenible.

☞ Expo

Plataforma que simplifica el build, preview y distribución de apps React Native. Imprescindible para empezar rápido.



React Native

Framework de Meta para crear apps nativas con componentes React. El mismo modelo mental que React web.



VS Code + Node.js

Editor recomendado con extensiones para React Native. Node.js es necesario para las herramientas de desarrollo.

Instalación del entorno (T01)

Lo que necesitas instalar

01

Node.js 18+

nodejs.org – Descarga la versión LTS.

02

VS Code + extensiones

React Native Tools, TypeScript, Prettier, ESLint.

03

Expo Go en el móvil

Disponible en App Store y Google Play. Te permite ver tu app en tiempo real.

04

Crear primer proyecto

```
npx create-expo-app PokeApp --template blank-typescript
```

Script automático

Si usas Windows, ejecuta el script de instalación automática incluido en los materiales del curso:

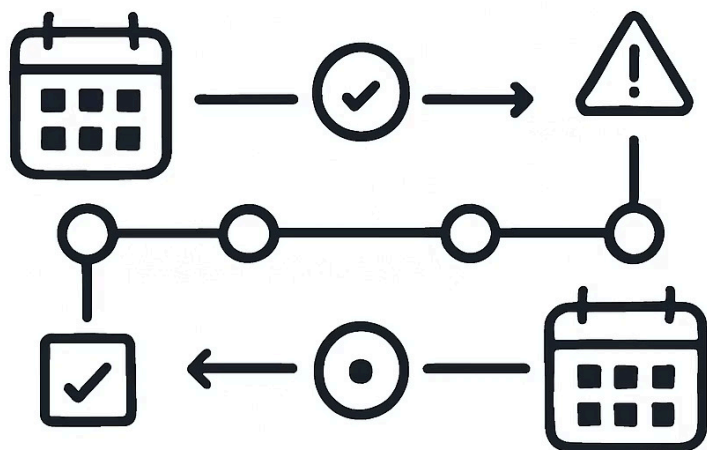
```
instalar-entorno.ps1
```

Guía paso a paso

Consulta la guía detallada en:

```
modulo-01/slides-t01.html
```

 **Objetivo T01:** app corriendo en tu móvil antes de terminar la sesión.



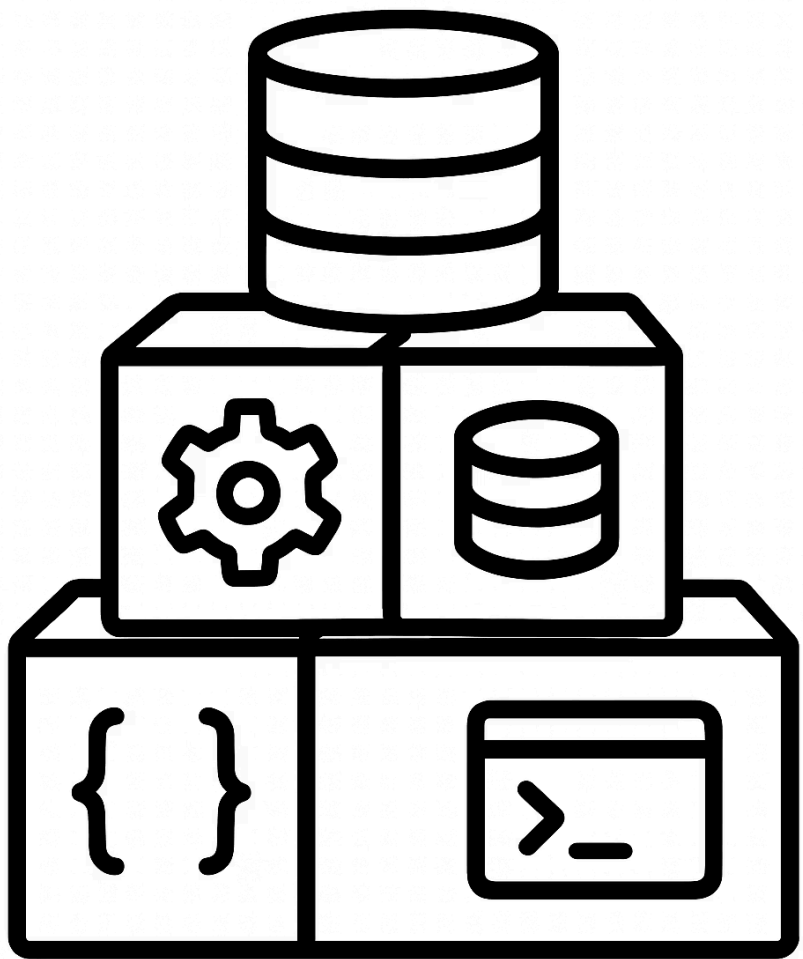
ESTRUCTURA

Estructura del curso

6 módulos · 17 topics · 1 app real

Visión general — 6 módulos

Módulo	Nombre	Topics	Resultado
1	Fundamentos	T01-T04	App corriendo + componentes propios
2	Listas y Navegación	T05-T07	3 pantallas navegables
3	Datos y API	T08-T10	Datos reales en pantalla
4	UI Avanzada	T11-T13	App con UI pulida y animaciones
5	Persistencia y Arquitectura	T14-T15	App con caché y estructura real
6	Calidad y Deploy	T16-T17	APK publicado con tests pasando



MÓDULO 1

Fundamentos

T01 — T04 · Entorno, JSX, Componentes y Estado

Módulo 1 — Fundamentos

1

T01 — Entorno y Expo

`create-expo-app`, Expo Go, estructura del proyecto, primer `Hello World`.

2

T02 — JSX y Estilos

`StyleSheet`, Flexbox para layouts, diferencias con CSS web.

3

T03 — Componentes y Props

Tipado TypeScript estricto, composición de componentes, reutilización.

4

T04 — Estado y Eventos

`useState`, `onPress`, Switch y formularios básicos.

Objetivo del módulo

Tendrás una app corriendo en tu máquina o en tu móvil **antes de terminar T01**.

Resultado final

App con componentes propios, props tipadas y estado interactivo funcionando.

Materiales

`modulo-01/slides-t01.html`

`modulo-01/slides-t02.html`

`modulo-01/slides-t03.html`

`modulo-01/slides-t04.html`

JSX: HTML en tu JavaScript

JSX es la sintaxis que permite escribir componentes visuales directamente en TypeScript. React Native lo convierte en vistas nativas reales de Android e iOS.

Componente básico en TypeScript

```
import { View, Text, StyleSheet }
  from 'react-native';

type Props = {
  nombre: string;
};

export function Saludo({ nombre }: Props) {
  return (
    <View style={styles.container}>
      <Text style={styles.texto}>
        ¡Hola, {nombre}!
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { padding: 16 },
  texto: { fontSize: 24 },
});
```

Conceptos clave de T01–T03

View

Equivalente a `<div>` en web. Contenedor base de cualquier layout.

Text

Todo el texto debe ir dentro de un `<Text>`, sin excepción.

StyleSheet

Estilos en JavaScript, similares a CSS pero con Flexbox por defecto.

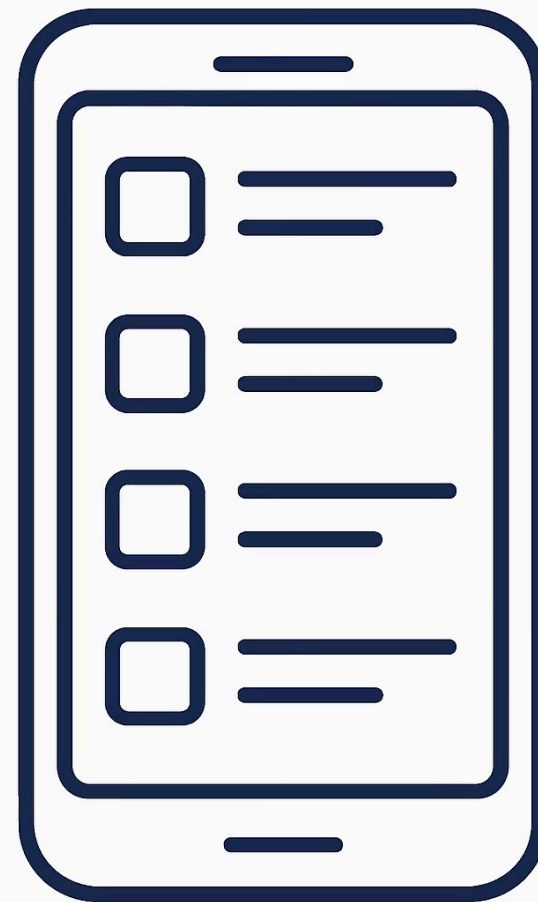
Props tipadas

TypeScript garantiza que nadie use el componente incorrectamente.

MÓDULO 2

Listas y Navegación

T05 — T07 · FlatList, Stack Navigator y Tab Navigator



Módulo 2 — Listas y Navegación

1

T05 — Listas con FlatList

Rendimiento optimizado con virtualización, `keyExtractor`, layouts de 2 columnas y `ListEmptyComponent`.

2

T06 — Stack Navigator

Navegación entre pantallas, parámetros tipados con TypeScript, configuración del header.

3

T07 — Tab Navigator

Pestañas inferiores, iconos con `@expo/vector-icons`, navegación combinada Stack + Tabs.

  Prerequisito crítico

Sin los fundamentos del M1 la navegación resulta muy difícil. Asegúrate de tener el M1 sólido antes de continuar.

Resultado final

App con **3 pestañas** y navegación fluida entre pantalla de lista y pantalla de detalle.

FlatList vs ScrollView

FlatList es la forma correcta de mostrar listas largas en React Native. A diferencia de ScrollView, solo renderiza los elementos visibles en pantalla, lo que lo hace extremadamente eficiente con cientos de items.

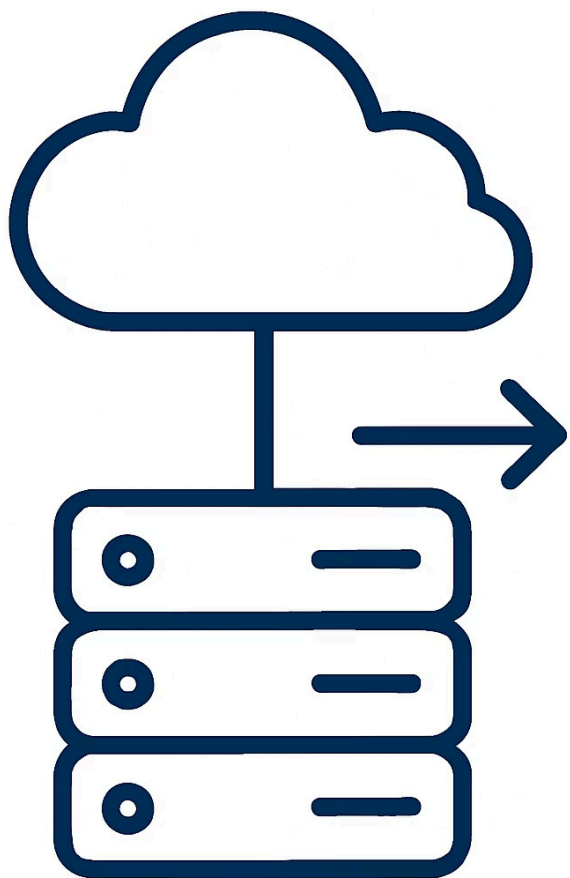
FlatList — Recomendado

```
// Solo renderiza lo visible
<FlatList
  data={pokemon}
  keyExtractor={({item}) =>
    item.id.toString()}
  renderItem={({ item }) => (
    <PokemonCard item={item} />
  )}
  numColumns={2}
/>
```

ScrollView — Solo listas cortas

```
// Renderiza TODO de una vez
<ScrollView>
  {pokemon.map(item => (
    <PokemonCard
      key={item.id}
      item={item}
    />
  ))}
</ScrollView>
```

 Regla práctica: si la lista tiene más de 20 elementos o carga desde API, usa siempre FlatList.



MÓDULO 3

Datos y API

T08 — T10 · Fetch, useEffect y Hooks personalizados

Módulo 3 — Datos y API

1

T08 — Fetch y async/await

Peticiones HTTP con la API nativa `fetch`, tipado estricto de la respuesta, manejo de errores de red.

2

T09 — `useEffect` y estados de carga

Ciclo de vida del componente, los tres estados esenciales: `loading`, `error` y `data`.

3

T10 — Hooks personalizados

Extraer lógica de negocio fuera del componente, prefijo `use`, reutilización entre pantallas.

PokéAPI en acción

```
GET https://pokeapi.co/
api/v2/pokemon?limit=20
```

Sin registro · Sin límite · Gratis.

Resultado final

Lista y detalle con datos reales de PokéAPI, con estados de carga y error correctamente gestionados.

El patrón loading / error / data

Toda petición a una API tiene tres estados posibles. Gestionarlos correctamente es la diferencia entre una app profesional y una que se queda en blanco o se cuelga.

loading: true

Muestra un `ActivityIndicator`. El usuario sabe que algo está pasando.

error: string

Muestra un mensaje claro y opcionalmente un botón de reintentar.

data: T[]

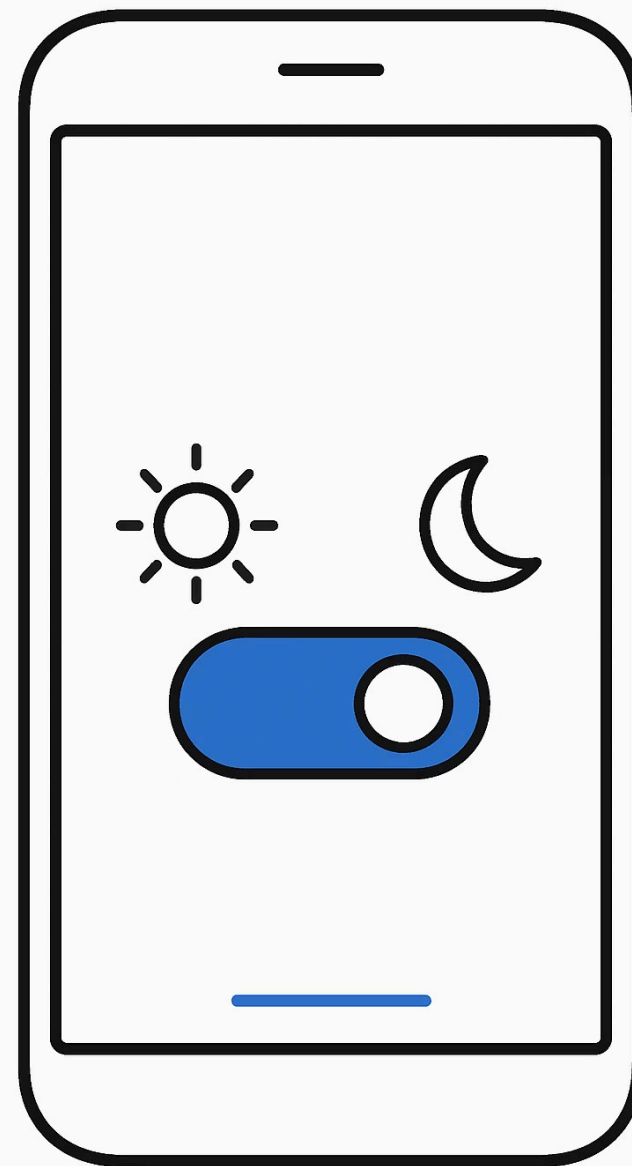
Renderiza la lista o el detalle con los datos recibidos de la API.

```
const [loading, setLoading] = useState(true);  
const [error, setError] = useState<string | null>(null);  
const [data, setData] = useState<Pokemon[]>([]);
```

MÓDULO 4

UI Avanzada

T11 — T13 · Context, Animaciones y Modal



Módulo 4 — UI Avanzada

1

T11 — Context y Temas

`createContext`, `useContext`, implementación de tema claro/oscuro conmutable en toda la app.

2

T12 — Animaciones

`Animated.Value`, interpolaciones, flip de tarjeta estilo CCG con SVG. La app cobra vida.

3

T13 — Modal

Modal nativo de React Native, backdrop semitransparente, `Pressable` para cerrar.

★ Punto de inflexión

Aquí la app empieza a parecerse a algo **real**. El usuario verá animaciones suaves, cambio de tema y modales nativos.

Referencia de código

```
PokeApp/src/components/
```

Resultado final

App con tema claro/oscuro, animación flip en tarjetas y modal de detalle.

Context API: estado global sin librerías

El Context de React permite compartir estado (como el tema de color) entre cualquier componente de la app sin necesidad de pasar props a través de múltiples niveles.

Crear y proveer el contexto

```
// ThemeContext.tsx
type ThemeContextType = {
  isDark: boolean;
  toggle: () => void;
};

export const ThemeContext =
  createContext<ThemeContextType>(
    { isDark: false, toggle: () => {} }
  );

export function ThemeProvider(
  { children }: { children: ReactNode }
) {
  const [isDark, setIsDark] =
    useState(false);
  return (
    <ThemeContext.Provider
      value={{ isDark,
        toggle: () => setIsDark(p => !p)
      }}>
      {children}
    </ThemeContext.Provider>
  );
}
```

Consumir el contexto

```
// En cualquier componente
function MiComponente() {
  const { isDark, toggle } =
    useContext(ThemeContext);

  return (
    <Switch
      value={isDark}
      onChange={toggle}
    />
  );
}
```

📄 Usaremos este patrón en T11 para el tema claro/oscuro. En M5 aprenderás a persistirlo con AsyncStorage.

MÓDULO 5

Persistencia y Arquitectura

T14 — T15 · AsyncStorage y estructura src/



Módulo 5 — Persistencia y Arquitectura

1

T14 — AsyncStorage

getItem, setItem, JSON.stringify/parse. Persistir preferencias del usuario entre sesiones (tema, favoritos).

2

T15 — Arquitectura src/

Tipos de dominio, mappers API→app, estructura de carpetas escalable. El código que aguanta el paso del tiempo.

Resultado final

App con preferencias persistidas entre sesiones y código organizado en capas bien definidas.

Estructura de referencia

```
src/  
├── api/      # fetch + mappers  
├── components/ # atómicos y  
│           # compuestos  
├── hooks/    # lógica  
│           # encapsulada  
├── screens/  # pantallas  
├── types/    # tipos de dominio  
└── constants/ # colores, config
```

Esta estructura sigue los principios de **separación de responsabilidades** usados en proyectos reales de producción.

Mappers: desacoplar la API de tu app

Un **mapper** transforma la respuesta cruda de la API en el tipo de dominio de tu app. Si la API cambia, solo tocas el mapper, no todos tus componentes.

Tipo de la API (externo)

```
// types/api.ts
type PokemonApiResponse = {
  id: number;
  name: string;
  height: number;
  sprites: {
    front_default: string;
  };
};
```

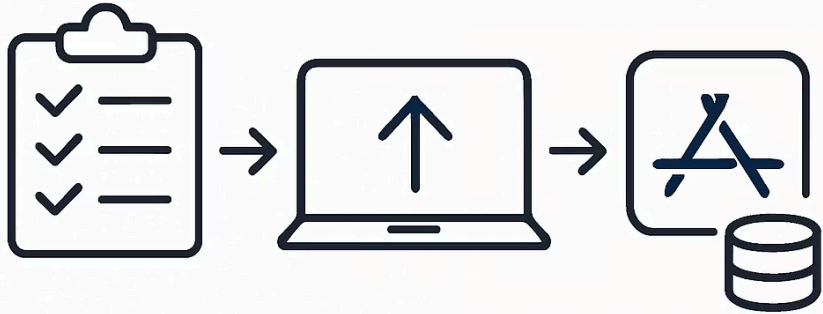
Tipo de dominio (interno)

```
// types/domain.ts
type Pokemon = {
  id: number;
  nombre: string;
  imageUrl: string;
  alturaMetros: number;
};
```

El mapper

```
// api/mappers.ts
export function toPokemon(
  raw: PokemonApiResponse
): Pokemon {
  return {
    id: raw.id,
    nombre: raw.name,
    imageUrl:
      raw.sprites.front_default,
    alturaMetros: raw.height / 10,
  };
}
```

✓ El resto de la app solo conoce `Pokemon`, nunca `PokemonApiResponse`. Esto es arquitectura limpia.



MÓDULO 6

Calidad y Deploy

T16 — T17 · Jest, Testing Library y EAS Build

Módulo 6 — Calidad y Deploy

1

T16 — Testing con Jest

Test de utilidad pura, test de hook con `renderHook` y test de componente con `Testing Library`. Garantía de que la app funciona.

2

T17 — EAS Build y Deploy

Perfiles `preview` y `production`, generación de APK compatible por QR, proceso completo de build en la nube.



Demo final

Cada alumno proyecta su app terminada. Objetivo: **orgullo del trabajo**.

Resultado final

App con tests pasando y APK publicado, instalable en cualquier dispositivo Android mediante QR.



Este es el momento en el que puedes decir: "**Construí una app móvil real.**"

Tu primer test en React Native

Los tests no son opcionales en proyectos reales. En T16 aprenderás a testear los tres tipos más comunes: utilidades puras, hooks y componentes.

Test de utilidad pura (más fácil)

```
// utils/format.test.ts
import { formatAltura } from './format';

test('convierte decímetros a metros', () => {
  expect(formatAltura(7)).toBe('0.7 m');
  expect(formatAltura(14)).toBe('1.4 m');
});
```

Test de componente

```
// components/PokemonCard.test.tsx
import { render, screen }
  from '@testing-library/react-native';

test('muestra el nombre del pokemon', () => {
  render(<PokemonCard
    nombre="bulbasaur" id={1} />);
  expect(
    screen.getByText('bulbasaur')
  ).toBeTruthy();
});
```

Utilidades puras

Funciones sin side effects.
Los tests más simples y
valiosos.

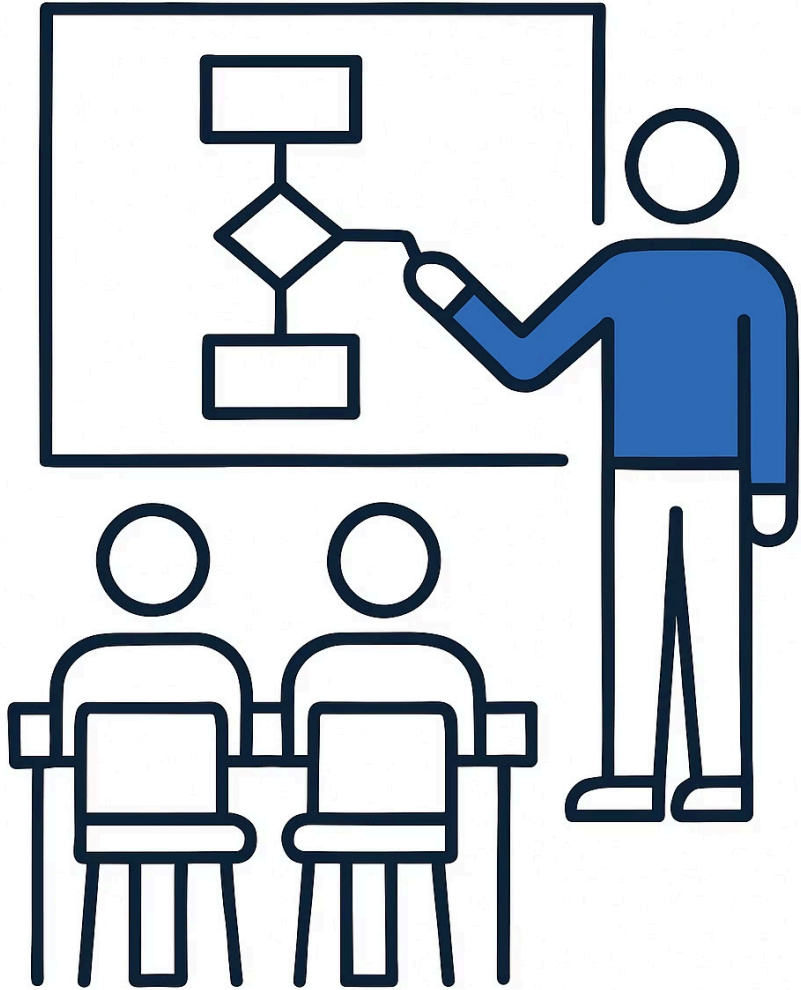
Hooks

`renderHook` de Testing
Library para aislar la lógica.

Componentes

Verificar que el usuario ve lo correcto en pantalla.

❗ Empieza siempre por los tests más simples. Un test que pasa es mejor que ninguno.



METODOLOGÍA

Metodología y evaluación

Cómo trabajaremos juntos

Metodología del curso

Learning by doing. Cada topic termina con código funcionando. No hay teoría sin práctica inmediata.

01

Lee las slides

Teoría y ejemplos de código comentados, disponibles en `modulo-XX/slides-tYY.html`.

03

Haz los ejercicios

Starter incluido. Si te atascas, la solución está disponible – pero inténtalo primero.

02

Sigue el código de referencia

PokeApp/ como guía paso a paso. Puedes consultarlo en cualquier momento.

04

Continúa a tu ritmo

No hay límite de tiempo por topic. El objetivo es entender, no correr.

⚠ Regla de oro: No copies el código de la solución sin intentarlo antes. Equivocarse es parte esencial del aprendizaje.

Materiales del curso

Archivo	Para quién	Contenido
<code>modulo-XX/slides-tYY.html</code>	Proyector / alumno	Slides del topic con teoría y ejemplos
Tu repo en GitHub	Alumno	Tu app creciendo módulo a módulo
<code>PokeApp/</code>	Referencia	Implementación completa de referencia del curso
<code>ejercicios/starter/</code>	Alumno	Punto de partida para cada ejercicio
<code>ejercicios/solucion/</code>	Alumno (si atasco)	Solución para consultar si te bloqueas

❗ Todos los materiales están disponibles en el repositorio del curso desde el primer día. No esperes a tener preguntas para explorarlos.

Criterios de evaluación



Entrega mínima para aprobar

- App que lista datos desde una API real
- Al menos una pantalla de detalle
- Navegación entre pantallas
- APK generado con EAS Build

i No se evalúa la estética sino la funcionalidad.
Una app fea que funciona > una app bonita que no compila.

Resumen: lo que vas a conseguir

Hoy entras sin saber...

✗ Qué es React Native

✗ Cómo hacer una app móvil

✗ Cómo conectar una API

Al terminar podrás...

✓ App publicada en tu móvil

✓ Modelo React / componentes

✓ Consumir cualquier API REST

✓ Seguir aprendiendo solo

¿Y después de este curso?

Este curso es el punto de partida. Una vez que domines los 6 módulos, el ecosistema React Native te ofrece muchos caminos para seguir creciendo como desarrollador.



Expo Router

File-based routing al estilo Next.js, pero para móvil. El futuro de la navegación en Expo.



Redux / Zustand

Gestión de estado global para apps complejas con múltiples fuentes de datos.



Testing con Detox

Tests end-to-end que simulan al usuario tocando la pantalla en un dispositivo real.



Publicación en stores

Play Store y App Store: firma, metadata, capturas y proceso completo de publicación.

¡A por ello! 🚀

→ Paso 1

T01 – Instala el entorno y lanza tu primera app con Expo Go.

→ Paso 2

Consulta el **código de referencia** de PokeApp/ cuando lo necesites.

→ Paso 3

¡Empieza cuando quieras! Tienes todos los materiales disponibles.

Carlos Rodríguez Contreras · carlos.mailhub@gmail.com · GitHub: karrocon

